

Section 11.2 of 10: Core Distributed Inference & KV Cache Management

Executive Overview

AI inference at scale demands careful orchestration of GPU resources, memory management, and batching strategies. The KV cache dominates memory usage during autoregressive generation, making efficient allocation critical for throughput. This section covers production inference patterns: tensor/pipeline parallelism, continuous batching, speculative decoding, and PagedAttention.

Math-heavy alert: KV cache sizing involves concrete memory arithmetic. Extra weight is given to worked numeric examples so you can derive these on a whiteboard. Parallelism trade-offs are explained via proof-by-contradiction where relevant.

11.2.1 GPU Resource Architecture

GPU Memory Hierarchy

CPU Host Memory (DRAM)	25-100 GB/s
PCIe Bus (CPU ↔ GPU)	16-64 GB/s
GPU High Bandwidth Memory (HBM)	1-3 TB/s
L2 Cache: 50-100 MB	5-10 TB/s
L1/Shared Memory: 256 KB	50+ TB/s
Tensor Cores / CUDA Cores	

GPU Selection Guide

GPU	VRAM	Bandwidth	Relative Cost/Token	Best Use Case
H100	80 GB	3.0 TB/s	High	Large models (70B+), production serving

GPU	VRAM	Bandwidth	Relative Cost/Token	Best Use Case
A100	40–80 GB	1.5 TB/s	Medium	Research, training, mid-size serving
A10G	24 GB	600 GB/s	Low	Small models (<13B), development
L4	16 GB	300 GB/s	Lowest	Edge inference, low throughput

When memory bandwidth bottlenecks matter: At inference time, the bottleneck is almost always memory bandwidth (reading model weights), not FLOPS. An H100 reads 3 TB/s vs A10G’s 600 GB/s — that’s a 5× throughput difference for memory-bound (small batch) inference.

11.2.2 KV Cache Memory Management

KV Cache Size — Worked Numeric Example

For Llama-2-70B (hidden size $H=8192$, 80 layers, FP16):

```
KV per token per layer =  $H \times 2$  (K and V)  $\times 2$  bytes (FP16)
                        =  $8192 \times 2 \times 2 = 32,768$  bytes = 32 KB
```

```
KV per token (all layers) = 32 KB  $\times$  80 = 2.56 MB per token
```

```
For 32K context:
  2.56 MB  $\times$  32,768 tokens = ~84 GB per request (!)
```

Proof by contradiction why this is a problem: Suppose you have an 80 GB H100. Model weights for Llama-70B consume ~140 GB FP16, so you already need 2 GPUs. With just 1 concurrent 32K-context request, the KV cache alone uses 84 GB — more than one GPU’s entire VRAM. Without PagedAttention or offloading, concurrent requests at long context is infeasible.

PagedAttention (vLLM)

Traditional contiguous allocation:

```
VRAM: [Request A: 500 tokens ██████████][Request B: 300 tokens ████████]
      [████fragmented████]
Problem: New 200-token request can't fit even though total free space > 200 tokens
```

PagedAttention:

```
Physical VRAM:      [Page 1: tokens 1-16][Page 2: tokens 17-32]...[Page N]
Virtual KV table:   Request A → [Page 1, Page 5, Page 9, ...]
                   Request B → [Page 2, Page 6, Page 10, ...]
```

Each page = 16 tokens (configurable). Pages are non-contiguous in physical memory but the virtual block table makes them appear contiguous to the attention kernel.

Benefits: - Near-zero memory fragmentation (waste < 4%) - Enables KV sharing across parallel beams in beam search - GPU→CPU paging: Evict cold pages to CPU RAM, reload on demand

Memory Strategy Comparison

Strategy	Memory Efficiency	Implementation Complexity	Context Switch Overhead
Contiguous	Poor (30–40% waste)	Simple	Slow (full copy)
PagedAttention	Excellent (<4% waste)	Complex	Fast (page-level)
FlashAttention	Good (recompute vs store)	Medium	N/A (computation optimization)

When to combine: Use PagedAttention for memory management + FlashAttention for computation efficiency. They address different problems (allocation vs. kernel efficiency).

11.2.3 Batching Strategies

Static vs. Continuous Batching

Static batching problem — worked example:

```
T=0ms:   Request A arrives (will generate 500 tokens)
T=2ms:   Request B arrives (will generate 10 tokens)
T=4ms:   Request C arrives (will generate 200 tokens)
         → Wait for batch of 3

T=10ms:  Batch starts
T=210ms: Request C finishes (200 tokens done)
         → But must wait for A (500 tokens)
T=510ms: Request A finishes, batch released
```

B and C waste 300ms and 100ms respectively waiting for A. Tail latency is driven by slowest request.

Continuous batching solution:

T=0ms: Request A starts (mid-batch iteration)
T=5ms: Request B arrives → inserted into next iteration
T=210ms: Request C finishes → slot freed, Request D inserted
(A still running; no wasted capacity)

Result: GPU utilization increases from ~60% (static) to ~90%+ (continuous).

Speculative Decoding — Numeric Speedup

Small draft model (8B): Generates 4 tokens in 5ms
Large target model (70B): Verifies 4 tokens in 15ms (parallel forward pass)

Without speculation:

70B model generates 4 tokens: $4 \times 15\text{ms} = 60\text{ms}$

With speculation (4 tokens drafted, 3 accepted on average):

Draft: 5ms + Verify: 15ms = 20ms for ~3 tokens

Effective speed: 3 tokens / 20ms = 0.15 tokens/ms vs 0.067 tokens/ms

Speedup: ~2.2×

When speculation fails: If draft acceptance rate < 50%, speculation adds overhead without meaningful gain. Monitor acceptance rate in production; disable if it drops below threshold.

Batching Strategy Comparison

Strategy	Throughput	p99 Latency	Complexity	Best For
Static batching	Good	Unpredictable (slowest request drives all)	Low	Batch jobs, offline inference
Continuous batching	Excellent	Predictable	Medium	Online serving, mixed request sizes
Speculative decoding	2–3×	Lower	High	Long-form generation, consistent domains

11.2.4 Model Parallelism

Tensor Parallelism — Worked Example

For a 4-way tensor parallel linear layer ($Y = XW$):

```
Full weight W: [4096 × 4096] (single GPU would need 32 MB FP16)
```

```
Split column-wise across 4 GPUs:
```

```
GPU 0:  $W_0 = [4096 \times 1024]$ 
```

```
GPU 1:  $W_1 = [4096 \times 1024]$ 
```

```
GPU 2:  $W_2 = [4096 \times 1024]$ 
```

```
GPU 3:  $W_3 = [4096 \times 1024]$ 
```

```
Each GPU computes partial result  $Y_{\text{partial}} = X \times W_n$ 
```

```
Then: AllReduce across GPUs to sum partial results
```

```
Final  $Y = Y_0 + Y_1 + Y_2 + Y_3$ 
```

Communication cost: AllReduce transfers $2 \times \text{batch_size} \times \text{seq_len} \times \text{hidden_size}$ bytes per layer. For long sequences, this dominates — use NVLink (600 GB/s) not PCIe (64 GB/s).

Pipeline Parallelism — Bubble Problem

```
Without micro-batching (4 GPUs, 4 layers each):
```

```
GPU 0: [Layer 0-3: microbatch 1] [idle] [idle] [idle]
GPU 1: [idle] [Layer 4-7: M1] [idle] [idle]
GPU 2: [idle] [idle] [Layer 8-11: M1] [idle]
GPU 3: [idle] [idle] [idle] [Layer 12-15: M1]
```

```
Pipeline bubble = 75% idle time at start ( $p-1/p$  where  $p=4$ )
```

```
With micro-batching (4 micro-batches):
```

```
GPU 0: [M1] [M2] [M3] [M4] [idle]
GPU 1: [---] [M1] [M2] [M3] [M4]
...
Bubble reduced to  $(p-1)/(p + m - 1)$  where  $m$  = micro-batches
```

Parallelism Selection Guide

Parallelism	Communication Overhead	Memory Efficiency	When to Use
Tensor (intra-node)	High (AllReduce per layer)	Excellent	Model too large for 1 GPU; use NVLink

Parallelism	Communication Overhead	Memory Efficiency	When to Use
Pipeline (inter-node)	Low (activation handoff)	Good	Multi-node; combine with micro-batches
Data parallel	Medium (gradient sync)	Poor (full model copy)	Small model, large dataset
3D combined	Variable	Excellent	70B+ models, cluster deployments

Interview Problems

Problem 1: Design Inference Service for Llama-3-70B at Scale

Scenario: 1,000 concurrent users, 32K context support, latency SLA of 2 seconds to first token.

Solution:

Hardware:

- 8× H100 80GB (tensor parallel, 2 nodes)
- Model weights: 140 GB ÷ 8 GPUs = 17.5 GB/GPU (fits with room for KV cache)

Memory allocation per GPU:

Model weights (FP16): 17.5 GB
KV cache (active window): ~20 GB
CUDA overhead: 5 GB
Remaining for PagedAttention pages: 37.5 GB

KV cache budget:

2.56 MB/token × active context per GPU
37.5 GB / 2.56 MB = ~14,600 tokens active per GPU

With CPU offloading (PagedAttention):

Cold pages → 512 GB CPU RAM (200,000+ token capacity)
LRU eviction: recently used tokens stay on GPU

Throughput math: - Continuous batching + speculative decoding → ~20 tokens/sec per request -
1,000 users × 20 tokens/sec = 20,000 tokens/sec total

Problem 2: Diagnosing Inference Latency Regression

Scenario: Inference p99 latency jumped from 500ms to 2000ms after doubling concurrent users from 50 to 100. What's wrong?

Diagnosis framework:

Hypothesis	Check	Fix
KV cache full → paging to CPU	Monitor GPU→CPU transfers	Reduce max_context or add GPU nodes
Batch size too large → compute bound	GPU utilization >95%	Cap batch size, use smaller model tier for simple queries
Memory fragmentation	vLLM memory stats	Restart with <code>--max-model-len</code> reduced
Draft model bottleneck	Speculation acceptance rate	Tune draft model or disable speculation

Key insight: Doubling users from 50→100 may cross the GPU memory threshold where PagedAttention starts evicting pages to CPU, adding 100–500ms per eviction cycle. The fix is to either add capacity or implement a request queue with max KV budget enforcement.

Problem 3: Cost Optimization for Mixed Workloads

Scenario: Your inference cluster handles both interactive chat (latency-sensitive) and batch summarization (throughput-sensitive). How do you allocate resources?

Solution:

```
Cluster split:
  Interactive pool (40% capacity):
    - Continuous batching, small batch sizes (8-16)
    - Speculative decoding enabled
    - SLA: p99 < 500ms first token

  Batch pool (60% capacity):
    - Static batching, large batch sizes (64-128)
    - No speculation (not worth overhead for long outputs)
    - SLA: throughput > 5,000 tokens/sec
```

```
Routing:
  if request.stream == true or timeout_seconds < 5:
    → interactive pool
  else:
    → batch pool (queue if full, ETA returned to caller)
```

This separation prevents batch jobs from monopolizing GPU memory and degrading interactive latency.